

实验一：渗透问题

一、实验目的

使用合并-查找（union-find）数据结构，编写程序通过蒙特卡罗模拟（Monte Carlo simulation）来估计渗透阈值的值。

二、实验内容

给定由随机分布的绝缘材料和金属材料构成的组合系统：金属材料占多大比例才能使组合系统成为电导体？给定一个表面有水的多孔渗水地形（或下面有油），水将在什么条件下能够通过底部排出（或油渗透到表面）？科学家们已经定义了一个称为渗透（percolation）的抽象过程来模拟这种情况。

如图 1.2.1 所示，我们使用 $N \times N$ 网格点来模型一个渗透系统。每个格点或是 open 格点或是 blocked 格点。一个 full site 是一个 open 格点，它可以通过一连串的邻近（左，右，上，下）open 格点连接到顶行的一个 open 格点。如果在底行中有一个 full site 格点，则称系统是渗透的。（对于绝缘/金属材料的例子，open 格点对应于金属材料，渗透系统有一条从顶行到底行的金属路径，且 full sites 格点导电。对于多孔物质示例，open 格点对应于空格，水可能流过，从而渗透系统使水充满 open 格点，自顶向下流动。

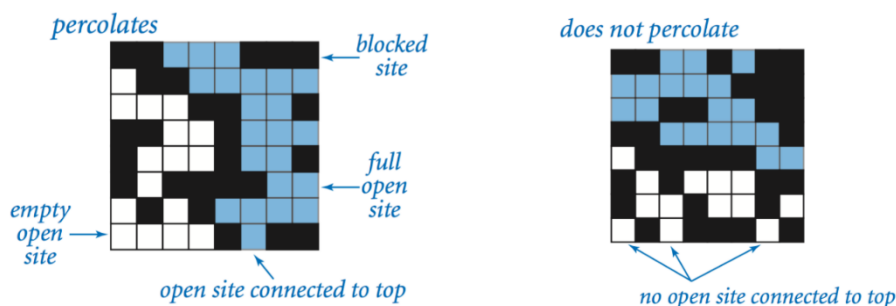


图 1.2.1

在上述模型中，我们将格点以空置概率 p 独立地设置为 open 格点（因此以概率 $1-p$ 被设置为 blocked 格点）。当 $p = 0$ 时，系统不会渗出；当 $p=1$ 时，系统渗透。图 1.2.2 显示了 20×20 随机网格（左）和 100×100 随机网格（右）的格点空置概率 p 与渗透概率。

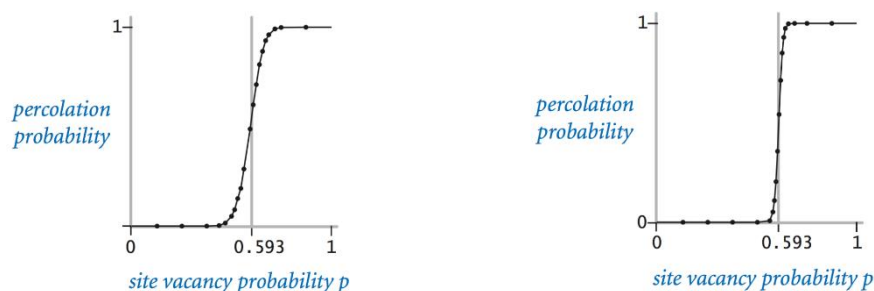


图 1.2.2

当 N 足够大时，存在阈值 p^* ，使得当 $p < p^*$ ，随机 $N \times N$ 网格几乎不会渗透，并且当 $p > p^*$ 时，随机 $N \times N$ 网格几乎总是渗透。尚未得出用于确定渗透阈值 p^* 的数学解。本次实验要求编写一个计算机程序来估计 p^* 。

三、实验思路

本次实验，我们使用蒙特卡洛模拟来估计渗透阈值。蒙特卡洛模拟的步骤如下：

- (1) 初始化所有格点为 blocked;
- (2) 重复以下操作直到系统渗出：
 - 在所有 blocked 的格点之间随机均匀选择一个格点 (row i, column j);
 - 设置这个格点 (row i, column j) 为 open 格点;

我们利用 open 格点的比例来估计系统渗透时的渗透阈值。例如，如果在 20×20 的网格中，如果我们测得当第 204 个格点被 open 时系统渗透，那么对渗透阈值的估计是 $204/400 = 0.51$ 。

通过重复该计算实验 T 次并对结果求平均值，可以获得了更准确的渗透阈值估计。同时，我们利用公式

$$\mu = \frac{x_1 + x_2 + \dots + x_T}{T}, \quad \sigma^2 = \frac{(x_1 - \mu)^2 + (x_2 - \mu)^2 + \dots + (x_T - \mu)^2}{T-1}$$

以及公式

$$\left[\mu - \frac{1.96\sigma}{\sqrt{T}}, \mu + \frac{1.96\sigma}{\sqrt{T}} \right]$$

可以求得渗透阈值提供 95% 置信区间。

四、具体实现

1、Percolation 数据类型。

模型化一个 Percolation 系统，创建含有以下 API 的数据类型 Percolation。

```
public class Percolation {
    public Percolation(int N) // 创建一个N-by-N的网格，所有位置初始时都是阻塞的
    public void open(int i, int j) // 如果位置（行i，列j）未打开，则打开它
    public boolean isOpen(int i, int j) // 位置（行i，列j）是否已打开？
    public boolean isFull(int i, int j) // 位置（行i，列j）是否已充满？
    public boolean percolates() // 系统是否渗透？
    public static void main(String[] args) // 测试客户端，可选
}
```

(1) Percolation 函数

```
public Percolation_NoWeighted(int n) {
    this.gridSize = n;
    this.siteStatus = new boolean[gridSize * gridSize + 2]; // 添加两个虚拟节点
    siteStatus[0] = true; // 初始化虚拟头部
    siteStatus[gridSize * gridSize + 1] = true; // 初始化虚拟尾部
    openCount = 0;
    unionFind = new QuickUnionUF(gridSize * gridSize + 2);
}
```

(2) Open 函数

```
public void open(int row, int col) {
    if (row < 1 || row > gridSize || col < 1 || col > gridSize) {
        throw new IndexOutOfBoundsException("index out of bounds");
    }

    if (isOpen(row, col)) {
        return; // 已经打开，直接返回
    }

    siteStatus[(row - 1) * gridSize + col] = true; // 打开此格点
    openCount++;

    // 与头部和尾部连接
    if (row == 1) {
        unionFind.union(0, (row - 1) * gridSize + col);
    }
    if (row == gridSize) {
        unionFind.union((row - 1) * gridSize + col, gridSize * gridSize + 1);
    }

    // 检查四个方向并进行合并
    int currentIndex = (row - 1) * gridSize + col;
    // 向左
    if (col > 1 && siteStatus[currentIndex - 1]) {
        unionFind.union(currentIndex, currentIndex - 1);
    }
    // 向右
    if (col < gridSize && siteStatus[currentIndex + 1]) {
        unionFind.union(currentIndex, currentIndex + 1);
    }
    // 向上
    if (row > 1 && siteStatus[currentIndex - gridSize]) {
        unionFind.union(currentIndex, currentIndex - gridSize);
    }
    // 向下
    if (row < gridSize && siteStatus[currentIndex + gridSize]) {
        unionFind.union(currentIndex, currentIndex + gridSize);
    }
}
```

(3) isOpen 函数

```
public boolean isOpen(int row, int col) {
    if (row < 1 || row > gridSize || col < 1 || col > gridSize) {
        throw new IndexOutOfBoundsException("index out of bounds");
    }
    return siteStatus[(row - 1) * gridSize + col];
}
```

(4) percolates 函数

```
public boolean percolates() {
    return unionFind.connected(0, gridSize * gridSize + 1);
}
```

2、PercolationStats 数据类型

我们创建数据类型 PercolationStats 来执行一系列计算实验，包含以下 API：

```
public class PercolationStats {
    public PercolationStats(int N, int T) // 在N-by-N网格上执行T次独立的计算实验
    public double mean() // 渗透阈值的样本均值
    public double stddev() // 渗透阈值的样本标准差
    public double confidenceLo() // 返回95%置信区间的下限
    public double confidenceHi() // 返回95%置信区间的上限
    public static void main(String[] args) // 测试客户端，如下所述
}
```

(1) PercolationStats 函数

```
public PercolationStats_NoWeighted(int n, int trials) {
    if (n <= 0 || trials <= 0) {
        throw new IllegalArgumentException("Illegal Argument");
    }
    gridLength = n;

    if (gridLength == 1)
    {
        average = 1;
        variance = Double.NaN;
        lowerConfidence = Double.NaN;
        upperConfidence = Double.NaN;
    }
    else
    {
        results = new double[trials];
        for (int i = 0; i < trials; i++) {
            Percolation_NoWeighted simulation = new Percolation_NoWeighted(n);
            while (!simulation.percolates()) {
                int row = StdRandom.uniform(n) + 1;
                int col = StdRandom.uniform(n) + 1;
                simulation.open(row, col);
            }
            results[i] = (double) simulation.OpenCount() / (n * n);
        }
        average = StdStats.mean(results);
        variance = StdStats.stddev(results);
        lowerConfidence = average - (1.96 * variance) / Math.sqrt(trials);
        upperConfidence = average + (1.96 * variance) / Math.sqrt(trials);
    }
}
```

(2) Mean 函数

```
public double mean()
{
    return average;
}
```

(3) Stddev 函数

```
public double stddev()
{
    return variance;
}
```

(4) confidenceLo 函数

```
public double confidenceLo()
{
    return lowerConfidence;
}
```

(5) confidenceHi 函数

```
public double confidenceHi()
{
    return upperConfidence;
}
```

(6) main 函数

```
public static void main(String[] args) {
    int n = 320;
    int trials = 1000;

    long startTime = System.nanoTime(); // 开始时间
    PercolationStats_NoWeighted statistics = new PercolationStats_NoWeighted(n,
trials);
    long endTime = System.nanoTime(); // 结束时间

    System.out.println("样本均值mean: " + statistics.mean());
    System.out.println("标准差stddev: " + statistics.stddev());
    System.out.println("置信区间confidence Low: " + statistics.confidenceLo());
    System.out.println("置信水平confidence High: " + statistics.confidenceHi());
    System.out.println("运行时间: " + (endTime-startTime) + " 纳秒");
}
```

五、 源代码

1、使用 quick-find 算法 (QuickFindUF.java from algs4.jar) 实现 Percolation 数据类型。

(1) Percolation_NoWeighted.java

```
1. import edu.princeton.cs.algs4.StdIn;
2. import edu.princeton.cs.algs4.QuickUnionUF;
3. import java.io.FileInputStream;
4. import java.io.FileNotFoundException;
```

```

5.
6. public class Percolation_NoWeighted {
7.     private QuickUnionUF unionFind;
8.     private int gridSize;
9.     private boolean[] siteStatus;
10.    private int openCount;
11.
12.    // 创建一个 n x n 的网格，所有格点初始为封闭状态
13.    public Percolation_NoWeighted(int n) {
14.        this.gridSize = n;
15.        this.siteStatus = new boolean[gridSize * gridSize + 2]; // 添加
        两个虚拟节点
16.        siteStatus[0] = true; // 初始化虚拟头部
17.        siteStatus[gridSize * gridSize + 1] = true; // 初始化虚拟尾部
18.        openCount = 0;
19.        unionFind = new QuickUnionUF(gridSize * gridSize + 2);
20.    }
21.
22.    // 打开指定的格点（行，列），如果尚未打开
23.    public void open(int row, int col) {
24.        if (row < 1 || row > gridSize || col < 1 || col > gridSize) {
25.            throw new IndexOutOfBoundsException("index out of bounds");
26.        }
27.
28.        if (isOpen(row, col)) {
29.            return; // 已经打开，直接返回
30.        }
31.
32.        siteStatus[(row - 1) * gridSize + col] = true; // 打开此格点
33.        openCount++;
34.
35.        // 与头部和尾部连接
36.        if (row == 1) {
37.            unionFind.union(0, (row - 1) * gridSize + col);
38.        }
39.        if (row == gridSize) {
40.            unionFind.union((row - 1) * gridSize + col, gridSize * grid
        Size + 1);
41.        }
42.
43.        // 检查四个方向并进行合并
44.        int currentIndex = (row - 1) * gridSize + col;
45.        // 向左

```

```

46.         if (col > 1 && siteStatus[currentIndex - 1]) {
47.             unionFind.union(currentIndex, currentIndex - 1);
48.         }
49.         // 向右
50.         if (col < gridSize && siteStatus[currentIndex + 1]) {
51.             unionFind.union(currentIndex, currentIndex + 1);
52.         }
53.         // 向上
54.         if (row > 1 && siteStatus[currentIndex - gridSize]) {
55.             unionFind.union(currentIndex, currentIndex - gridSize);
56.         }
57.         // 向下
58.         if (row < gridSize && siteStatus[currentIndex + gridSize]) {
59.             unionFind.union(currentIndex, currentIndex + gridSize);
60.         }
61.     }
62.
63.     // 检查指定的格点是否开放
64.     public boolean isOpen(int row, int col) {
65.         if (row < 1 || row > gridSize || col < 1 || col > gridSize) {
66.             throw new IndexOutOfBoundsException("index out of bounds");
67.         }
68.         return siteStatus[(row - 1) * gridSize + col];
69.     }
70.
71.     // 检查系统是否渗透
72.     public boolean percolates() {
73.         return unionFind.connected(0, gridSize * gridSize + 1);
74.     }
75.
76.     // 返回现有的 openCount
77.     public int OpenCount() {
78.         return openCount;
79.     }
80.
81.     public static void main(String[] args) {
82.         try {
83.             FileInputStream fileInput = new FileInputStream("input.txt"
84.             );
85.             System.setIn(fileInput);
86.         } catch (FileNotFoundException e) {
87.             e.printStackTrace();
88.         }

```

```

88.         int n = StdIn.readInt();
89.         Percolation_NoWeighted simulation = new Percolation_NoWeighted(
           n);
90.         while (!StdIn.isEmpty()) {
91.             int row = StdIn.readInt();
92.             int col = StdIn.readInt();
93.             simulation.open(row, col);
94.         }
95.         System.out.println(simulation.percolates());
96.     }
97. }

```

(2) PercolationStats_NoWeighted.java

```

1. import edu.princeton.cs.algs4.StdRandom;
2. import edu.princeton.cs.algs4.StdStats;
3.
4. public class PercolationStats_NoWeighted {
5.     private int gridLength;
6.     private double average; //平均值
7.     private double variance; //方差
8.     private double lowerConfidence; //置信区间下限
9.     private double upperConfidence; //置信区间上限
10.    private double[] results;
11.
12.    // 在 n x n 网格上执行独立实验
13.    public PercolationStats_NoWeighted(int n, int trials) {
14.        if (n <= 0 || trials <= 0) {
15.            throw new IllegalArgumentException("Illegal Argument")
16.        }
17.        gridLength = n;
18.
19.        if (gridLength == 1)
20.        {
21.            average = 1;
22.            variance = Double.NaN;
23.            lowerConfidence = Double.NaN;
24.            upperConfidence = Double.NaN;
25.        }
26.        else
27.        {
28.            results = new double[trials];
29.            for (int i = 0; i < trials; i++) {

```

```

30.             Percolation_NoWeighted simulation = new Percolatio
n_NoWeighted(n);
31.             while (!simulation.percolates()) {
32.                 int row = StdRandom.uniform(n) + 1;
33.                 int col = StdRandom.uniform(n) + 1;
34.                 simulation.open(row, col);
35.             }
36.             results[i] = (double) simulation.OpenCount() / (n
* n);
37.         }
38.         average = StdStats.mean(results);
39.         variance = StdStats.stddev(results);
40.         lowerConfidence = average - (1.96 * variance) / Math.s
qrt(trials);
41.         upperConfidence = average + (1.96 * variance) / Math.s
qrt(trials);
42.     }
43. }
44.
45. public double mean()
46. {
47.     return average;
48. }
49.
50. public double stddev()
51. {
52.     return variance;
53. }
54.
55. public double confidenceLo()
56. {
57.     return lowerConfidence;
58. }
59.
60. public double confidenceHi()
61. {
62.     return upperConfidence;
63. }
64.
65.
66. public static void main(String[] args) {
67.     int n = 320;
68.     int trials = 1000;
69.

```

```

70.         long startTime = System.nanoTime(); // 开始时间
71.         PercolationStats_NoWeighted statistics = new PercolationStats_NoWeighted(n, trials);
72.         long endTime = System.nanoTime(); // 结束时间
73.
74.         System.out.println("样本均值
mean: " + statistics.mean());
75.         System.out.println("标准差
stddev: " + statistics.stddev());
76.         System.out.println("置信区间
confidence Low: " + statistics.confidenceLo());
77.         System.out.println("置信水平
confidence High: " + statistics.confidenceHi());
78.         System.out.println("运行时间: " + (endTime -
startTime) + " 纳秒");
79.     }
80. }

```

2、使用 weighted quick-union 算法 (WeightedQuickUnionUF.java from algs4.jar) 实现 Percolation 数据类型。

(1) Percolation_Weighted.java

```

1. import edu.princeton.cs.algs4.StdIn;
2. import edu.princeton.cs.algs4.WeightedQuickUnionUF;
3. import java.io.FileInputStream;
4. import java.io.FileNotFoundException;
5.
6. public class Percolation_Weighted {
7.     private WeightedQuickUnionUF unionFind;
8.     private int gridSize;
9.     private boolean[] siteStatus;
10.    private int openCount;
11.
12.    // 创建一个 n x n 的网格，所有格点初始为封闭状态
13.    public Percolation_Weighted(int n) {
14.        this.gridSize = n;
15.        this.siteStatus = new boolean[gridSize * gridSize + 2]; //
        添加两个虚拟节点
16.        siteStatus[0] = true; // 初始化虚拟头部
17.        siteStatus[gridSize * gridSize + 1] = true; // 初始化虚拟尾
        部
18.        openCount = 0;
19.        unionFind = new WeightedQuickUnionUF(gridSize * gridSize + 2
        );
20.    }

```

```

21.
22.     // 打开指定的格点（行，列），如果尚未打开
23.     public void open(int row, int col) {
24.         if (row < 1 || row > gridSize || col < 1 || col > gridSize)
25.         {
26.             throw new IndexOutOfBoundsException("index out of bounds
27.             ");
28.         }
29.         if (isOpen(row, col)) {
30.             return; // 已经打开，直接返回
31.         }
32.         siteStatus[(row - 1) * gridSize + col] = true; // 打开此格
33.         openCount++;
34.
35.         // 与头部和尾部连接
36.         if (row == 1) {
37.             unionFind.union(0, (row - 1) * gridSize + col);
38.         }
39.         if (row == gridSize) {
40.             unionFind.union((row - 1) * gridSize + col, gridSize * g
41.             ridSize + 1);
42.         }
43.
44.         // 检查四个方向并进行合并
45.         int currentIndex = (row - 1) * gridSize + col;
46.         // 向左
47.         if (col > 1 && siteStatus[currentIndex - 1]) {
48.             unionFind.union(currentIndex, currentIndex - 1);
49.         }
50.         // 向右
51.         if (col < gridSize && siteStatus[currentIndex + 1]) {
52.             unionFind.union(currentIndex, currentIndex + 1);
53.         }
54.         // 向上
55.         if (row > 1 && siteStatus[currentIndex - gridSize]) {
56.             unionFind.union(currentIndex, currentIndex - gridSize);
57.         }
58.         // 向下
59.         if (row < gridSize && siteStatus[currentIndex + gridSize]) {

```

```

59.         unionFind.union(currentIndex, currentIndex + gridSize);
60.     }
61. }
62.
63. // 检查指定的格点是否开放
64. public boolean isOpen(int row, int col) {
65.     if (row < 1 || row > gridSize || col < 1 || col > gridSize)
66.     {
67.         throw new IndexOutOfBoundsException("index out of bounds");
68.     }
69.     return siteStatus[(row - 1) * gridSize + col];
70. }
71. // 检查系统是否渗透
72. public boolean percolates() {
73.     return unionFind.connected(0, gridSize * gridSize + 1);
74. }
75.
76. public int OpenCount() {
77.     return openCount;
78. }
79.
80. public static void main(String[] args) {
81.     try {
82.         FileInputStream fileInput = new FileInputStream("input.txt");
83.         System.setIn(fileInput);
84.     } catch (FileNotFoundException e) {
85.         e.printStackTrace();
86.     }
87.     int n = StdIn.readInt();
88.     Percolation_Weighted simulation = new Percolation_Weighted(n);
89.     while (!StdIn.isEmpty()) {
90.         int row = StdIn.readInt();
91.         int col = StdIn.readInt();
92.         simulation.open(row, col);
93.     }
94.     System.out.println(simulation.percolates());
95. }
96. }

```

(2) PercolationStats_Weighted.java

```
1. import edu.princeton.cs.algs4.StdRandom;
2. import edu.princeton.cs.algs4.StdStats;
3.
4. public class PercolationStats_Weighted {
5.     private int gridLength;
6.     private double average; //平均值
7.     private double variance; //方差
8.     private double lowerConfidence; //置信区间下限
9.     private double upperConfidence; //置信区间上限
10.    private double[] results;
11.
12.    // 在 n x n 网格上执行独立实验
13.    public PercolationStats_Weighted(int n, int trials) {
14.        if (n <= 0 || trials <= 0) {
15.            throw new IllegalArgumentException("Illegal Argument");
16.        }
17.        gridLength = n;
18.
19.        if (gridLength == 1)
20.        {
21.            average = 1;
22.            variance = Double.NaN;
23.            lowerConfidence = Double.NaN;
24.            upperConfidence = Double.NaN;
25.        }
26.        else
27.        {
28.            results = new double[trials];
29.            for (int i = 0; i < trials; i++) {
30.                Percolation_Weighted simulation = new Percolation_Weighted(n);
31.                while (!simulation.percolates()) {
32.                    int row = StdRandom.uniform(n) + 1;
33.                    int col = StdRandom.uniform(n) + 1;
34.                    simulation.open(row, col);
35.                }
36.                results[i] = (double) simulation.OpenCount() / (n * n);
37.            }
38.            average = StdStats.mean(results);
39.            variance = StdStats.stddev(results);
```

```

40.         lowerConfidence = average - (1.96 * variance) / Math.sqrt(
            t(trials));
41.         upperConfidence = average + (1.96 * variance) / Math.sqrt(
            t(trials));
42.     }
43. }
44.
45. public double mean()
46. {
47.     return average;
48. }
49.
50. public double stddev()
51. {
52.     return variance;
53. }
54.
55. public double confidenceLo()
56. {
57.     return lowerConfidence;
58. }
59.
60. public double confidenceHi()
61. {
62.     return upperConfidence;
63. }
64.
65.
66. public static void main(String[] args) {
67.     int n = 320;
68.     int trials = 1000;
69.
70.     long startTime = System.nanoTime(); // 开始时间
71.     PercolationStats_Weighted statistics = new PercolationStats_
        Weighted(n, trials);
72.     long endTime = System.nanoTime(); // 结束时间
73.
74.     System.out.println("样本均值 mean: " + statistics.mean());
75.     System.out.println("标准差
        stddev: " + statistics.stddev());
76.     System.out.println("置信区间
        confidence Low: " + statistics.confidenceLo());
77.     System.out.println("置信水平
        confidence High: " + statistics.confidenceHi());

```

```

78.         System.out.println("运行时间: " + (endTime-startTime) + " 纳
           秒");
79.     }
80. }

```

六、实验结果及分析

1、实验结果记录如下:

N		10	20	40	80	160	320
Quick-Find	mean	0.5906	0.5913	0.5905	0.5932	0.5924	0.5926
	stddev	0.0741	0.0473	0.0300	0.0190	0.0116	0.0071
	confidenceLow	0.5861	0.5884	0.5886	0.5921	0.5917	0.5921
	confidenceHigh	0.5952	0.5942	0.5923	0.5944	0.5931	0.5930
	time(ms)	14	36	131	617	3713	25611
Weighted Quick-Union	mean	0.5882	0.5913	0.5911	0.5932	0.5927	0.5924
	stddev	0.0768	0.0487	0.0303	0.0189	0.0113	0.0067
	confidenceLow	0.5835	0.5883	0.5892	0.5920	0.5920	0.5920
	confidenceHigh	0.5930	0.5943	0.5929	0.5944	0.5934	0.5929
	time(ms)	10	26	71	257	1021	4134
注: T=1000							

2、实验分析:

- (1) 对于 Quick-Find 算法: 我们知道, Quickfind 应该是 $O(N^2)$ 的算法, 设: $\text{time} = a * N^b$, 其中 a 和 b 是常数。取对数, 将问题转化为线性回归的形式: $\log(\text{time}) = \log(a) + b * \log(N)$ 。

计算 $\log(N)$ 和 $\log(\text{time})$:

$\log(10) \approx 1.0, \log(14) \approx 1.146$
 $\log(20) \approx 1.301, \log(36) \approx 1.556$
 $\log(40) \approx 1.602, \log(131) \approx 2.117$
 $\log(80) \approx 1.903, \log(617) \approx 2.790$
 $\log(160) \approx 2.204, \log(3713) \approx 3.569$
 $\log(320) \approx 2.505, \log(25611) \approx 4.408$

使用这些数据点进行线性回归, 可以得到近似的线性方程:

斜率(b) ≈ 2.78
 截距($\log(a)$) ≈ -1.54

$\text{time} \approx e^{-1.54} * N^{2.78}$

- (2) 对于 Weighted Quick-Union 算法: 使用相同的表示法:

计算 $\log(N)$ 和 $\log(\text{time})$:

$\log(10) \approx 1.0, \log(10) \approx 1.000$
 $\log(20) \approx 1.301, \log(26) \approx 1.415$
 $\log(40) \approx 1.602, \log(71) \approx 1.851$
 $\log(80) \approx 1.903, \log(257) \approx 2.409$
 $\log(160) \approx 2.204, \log(1021) \approx 3.009$
 $\log(320) \approx 2.505, \log(4134) \approx 3.616$

使用这些数据点进行线性回归，可以得到近似的线性方程：

斜率(b) ≈ 2.08

截距($\log(a)$) ≈ -1.14

$\text{time} \approx e^{-1.14 * N^{2.08}}$

七、 实验总结

本次实验的主要目的是通过编程实践，深入理解和应用合并-查找（union-find）数据结构来解决渗透问题，并通过蒙特卡罗模拟估计渗透阈值。

在实验中，我们将渗透问题转化为连通性问题，巧妙地使用虚拟的顶部和底部节点，将系统渗透等价于这两节点之间的连通性，简化了问题处理。由于渗透概率 p 难以直接控制，我们采用蒙特卡罗模拟的方法，随机选择格点打开直到系统发生渗透，并通过计算 open 格点数与总格点数的比例来估计渗透阈值 p^* 。为了确保估计的可靠性，我们进行了多次独立实验（ T 次），并计算了样本均值、标准差及 95% 置信区间。掌握了这些关键概念后，我实现了 Percolation 类和 PercolationStats 类，编写了完整的程序，并对运行时间和内存占用进行了分析，进一步加深了对算法效率的理解。

实验二：排序算法对比

一、实验目的

在计算机上针对不同输入规模数据进行实验,对比排序算法的时间及空间占用性能。

二、实验内容

实现插入排序(Insertion Sort, IS),自顶向下归并排序(Top-down Mergesort, TDM),自底向上归并排序(Bottom-up Mergesort, BUM),随机快速排序(Random Quicksort, RQ),Dijkstra 3-路划分快速排序(Quicksort with Dijkstra 3-way Partition, QD3P)。在你的计算机上针对不同输入规模数据进行实验,对比上述排序算法的时间及空间占用性能。要求对于每次输入运行 10 次,记录每次时间/空间占用,取平均值。

三、实验方法

1、测试时间

时间占用使用 JAVA 自带的 `system.nanoTime()` 函数即可。`system.nanoTime()` 函数返回正在运行的 Java 进程的高精度时间源的当前值,以纳秒为单位。

我们只需在排序进程运行前后各调用一次 `system.nanoTime()` 函数得到其起始时间和结束时间,两者做差即可求出时间占用。如下:

```
● ● ●  
  
long startTime = System.nanoTime();  
  
/* 排序进程 */  
  
long endTime = System.nanoTime();  
  
StdOut.println((endTime - startTime) / 1000000.0);
```

2、测试空间

空间计算需要使用 JAVA runtime 类实例的两个对象函数,即 `runtime.freememory()` 和 `runtime.totalmemory()`。

`runtime.freememory()` 返回 JVM 的空闲内存量,以字节为单位;

`runtime.totalmemory()` 返回 JVM 已经从操作系统那里挖过来的内存大小,即进程当时所被分配的所有内存,以字节为单位;

由此我们可以得知,`runtime.totalmemory() - runtime.freememory()` 即为当时所占用的内存大小。如下:

```
● ● ●  
  
long startMemo = run.totalMemory() - run.freeMemory();  
  
/* 排序进程 */  
  
long endMemo = run.totalMemory() - run.freeMemory();  
  
StdOut.println((endMemo - startMemo) / 1024);
```

四、 源代码

1、 InsertionSort.java

```
1. import edu.princeton.cs.algs4.StdOut;
2.
3. public class InsertionSort {
4.
5.     // 插入排序的主要方法
6.     public static void sort(Comparable[] a) {
7.         int N = a.length;
8.         for (int i = 1; i < N; i++) {
9.             Comparable v = a[i];
10.            int j = i;
11.            while (j > 0 && less(v, a[j - 1])) {
12.                a[j] = a[j - 1];
13.                j--;
14.            }
15.            a[j] = v;
16.        }
17.    }
18.
19.    // 辅助方法，用于比较元素 v 和 w
20.    private static boolean less(Comparable v, Comparable w) {
21.        return v.compareTo(w) < 0;
22.    }
23.
24.    // 测试时间
25.    public static double time(Comparable[] a) {
26.        StdOut.println("插入排序:");
27.        for (int i = 0; i < 10; i++) {
28.            Comparable[] A = a.clone();
29.
30.            //         for (int j = 0; j < a.length; j++) {
31.            //             System.out.print(A[j] + " "); // 打印原数组 a 的每个元
32.            //             素
33.            //         }
34.            //         System.out.print(" ---- ");
35.
36.            long StartTime = System.nanoTime();
37.            InsertionSort.sort(A);
38.            long endTime = System.nanoTime();
39.            //         for (int j = 0; j < a.length; j++) {
```

```

40. //          System.out.print(A[j] + " "); // 打印原数组 a 的每个元
    素
41. //          }
42.
43.          StdOut.println((endTime - StartTime) / 1000000.0);
44.      }
45.      StdOut.println(" ");
46.      return 0;
47.  }
48.
49.  // 测试空间
50.  public static double memo(Comparable[] a) {
51.      StdOut.println("插入排序:");
52.      for (int i = 0; i < 10; i++) {
53.          Comparable[] B = a.clone();
54.
55.          Runtime run = Runtime.getRuntime();
56.          run.gc();
57.          long startMemo = run.totalMemory() - run.freeMemory();
58.          InsertionSort.sort(B);
59.          long endMemo = run.totalMemory() - run.freeMemory();
60.
61.          StdOut.println((endMemo - startMemo) / 1024);
62.      }
63.      return 0;
64.  }
65. }

```

2、TopDownMergesort. java

```

1. import edu.princeton.cs.algs4.StdOut;
2.
3. public class TopDownMergesort {
4.
5.     // 主排序方法，接收一个数组并对其进行排序
6.     public static void sort(Comparable[] a) {
7.         Comparable[] aux = new Comparable[a.length]; // 辅助数组，用于归并
8.         sort(a, aux, 0, a.length - 1); // 调用递归的私有排序方法
9.     }
10.
11.     // 私有的递归排序方法
12.     private static void sort(Comparable[] a, Comparable[] aux, int lo, i
        nt hi) {
13.         if (hi <= lo) {
14.             return; // 基本情况，子数组只有一个元素或为空

```

```

15.     }
16.     int mid = lo + (hi - lo) / 2; // 计算中点，将数组分成两半
17.     sort(a, aux, lo, mid); // 递归排序左半部分
18.     sort(a, aux, mid + 1, hi); // 递归排序右半部分
19.     merge(a, aux, lo, mid, hi); // 归并两个已排序的半部分
20. }
21.
22. // 归并操作
23. private static void merge(Comparable[] a, Comparable[] aux, int lo,
    int mid, int hi) {
24.     for (int k = lo; k <= hi; k++) {
25.         aux[k] = a[k]; // 将原数组的元素复制到辅助数组
26.     }
27.     int i = lo, j = mid + 1;
28.     for (int k = lo; k <= hi; k++) {
29.         if (i > mid) {
30.             a[k] = aux[j++]; // 左半部分用完，取右半部分的元素
31.         } else if (j > hi) {
32.             a[k] = aux[i++]; // 右半部分用完，取左半部分的元素
33.         } else if (less(aux[j], aux[i])) {
34.             a[k] = aux[j++]; // 右半部分的元素较小，取右半部分的元素
35.         } else {
36.             a[k] = aux[i++]; // 左半部分的元素较小或相等，取左半部分的元
    素
37.         }
38.     }
39. }
40.
41. // 辅助方法，用于比较两个元素
42. private static boolean less(Comparable v, Comparable w) {
43.     return v.compareTo(w) < 0;
44. }
45.
46. // 测试时间
47. public static double time(Comparable[] a) {
48.     StdOut.println("自顶向下归并排序:");
49.     for (int i = 0; i < 10; i++) {
50.         Comparable[] A = a.clone();
51.
52.         long StartTime = System.nanoTime();
53.         TopDownMergesort.sort(A);
54.         long endTime = System.nanoTime();
55.
56.         StdOut.println((endTime - StartTime) / 1000000.0);

```

```

57.     }
58.     StdOut.println(" ");
59.     return 0;
60. }
61.
62. // 测试空间
63. public static double memo(Comparable[] a) {
64.     StdOut.println("自顶向下归并排序:");
65.     for (int i = 0; i < 10; i++) {
66.         Comparable[] B = a.clone();
67.
68.         Runtime run = Runtime.getRuntime();
69.         run.gc();
70.         long startMemo = run.totalMemory() - run.freeMemory();
71.         TopDownMergesort.sort(B);
72.         long endMemo = run.totalMemory() - run.freeMemory();
73.
74.         StdOut.println((endMemo - startMemo) / 1024);
75.     }
76.     return 0;
77. }
78. }

```

3、BottomUpMergesort.java

```

1. import edu.princeton.cs.algs4.StdOut;
2.
3. public class BottomUpMergesort {
4.
5.     // 主排序方法，接收一个数组并对其进行排序
6.     public static void sort(Comparable[] a) {
7.         Comparable[] aux = new Comparable[a.length]; // 辅助数组，用于归并
           操作
8.         int N = a.length;
9.
10.        // 从大小为 1 的子数组开始，逐步增加子数组的大小
11.        for (int sz = 1; sz < N; sz = sz + sz) {
12.            // 从数组的开始位置开始，逐步归并更大的子数组
13.            for (int lo = 0; lo < N - sz; lo += sz + sz) {
14.                int mid = Math.min(lo + sz - 1, N - 1);
15.                int hi = Math.min(lo + sz + sz - 1, N - 1);
16.                merge(a, aux, lo, mid, hi);
17.            }
18.        }
19.    }

```

```
20.
21.     // 归并操作
22.     private static void merge(Comparable[] a, Comparable[] aux, int lo,
    int mid, int hi) {
23.         // 复制数组元素到辅助数组
24.         for (int k = lo; k <= hi; k++) {
25.             aux[k] = a[k];
26.         }
27.         int i = lo, j = mid + 1;
28.         for (int k = lo; k <= hi; k++) {
29.             if (i > mid) {
30.                 a[k] = aux[j++];
31.             } else if (j > hi) {
32.                 a[k] = aux[i++];
33.             } else if (less(aux[j], aux[i])) {
34.                 a[k] = aux[j++];
35.             } else {
36.                 a[k] = aux[i++];
37.             }
38.         }
39.     }
40.
41.     // 辅助方法，用于比较两个元素
42.     private static boolean less(Comparable v, Comparable w) {
43.         return v.compareTo(w) < 0;
44.     }
45.
46.     // 测试时间
47.     public static double time(Comparable[] a) {
48.         StdOut.println("自底向上归并排序:");
49.         for (int i = 0; i < 10; i++) {
50.             Comparable[] A = a.clone();
51.
52.             long StartTime = System.nanoTime();
53.             BottomUpMergesort.sort(A);
54.             long endTime = System.nanoTime();
55.
56.             StdOut.println((endTime - StartTime) / 1000000.0);
57.         }
58.         StdOut.println(" ");
59.         return 0;
60.     }
61.
62.     // 测试空间
```

```

63.     public static double memo(Comparable[] a) {
64.         StdOut.println("自底向上归并排序:");
65.         for (int i = 0; i < 10; i++) {
66.             Comparable[] B = a.clone();
67.
68.             Runtime run = Runtime.getRuntime();
69.             run.gc();
70.             long startMemo = run.totalMemory() - run.freeMemory();
71.             BottomUpMergesort.sort(B);
72.             long endMemo = run.totalMemory() - run.freeMemory();
73.
74.             StdOut.println((endMemo - startMemo) / 1024);
75.         }
76.         return 0;
77.     }
78.
79. }

```

4、RandomQuicksort.java

```

1. import java.util.Random;
2. import edu.princeton.cs.algs4.StdOut;
3.
4. public class RandomQuicksort {
5.
6.     private static final Random random = new Random();
7.
8.     // 主排序方法，接收一个数组并对其进行排序
9.     public static void sort(Comparable[] a) {
10.         shuffle(a); // 随机打乱数组
11.         sort(a, 0, a.length - 1); // 调用递归的私有排序方法
12.     }
13.
14.     // 随机打乱数组
15.     private static void shuffle(Comparable[] a) {
16.         for (int i = 0; i < a.length; i++) {
17.             int r = i + random.nextInt(a.length - i); // 随机选择一个元素
18.             exch(a, i, r); // 交换当前元素和随机选择的元素
19.         }
20.     }
21.
22.     // 私有的递归排序方法
23.     private static void sort(Comparable[] a, int lo, int hi) {
24.         if (hi <= lo) {

```

```
25.         return; // 基本情况，子数组只有一个元素或为空
26.     }
27.     int j = partition(a, lo, hi); // 划分数组并返回基准元素的位置
28.     sort(a, lo, j - 1); // 递归排序左半部分
29.     sort(a, j + 1, hi); // 递归排序右半部分
30. }
31.
32. // 划分操作
33. private static int partition(Comparable[] a, int lo, int hi) {
34.     Comparable v = a[lo]; // 选择基准元素
35.     int i = lo, j = hi + 1;
36.     while (true) {
37.         while (less(a[++i], v)){
38.             if(i == hi) break;
39.         } // 从左向右找到第一个大于等于基准的元素
40.
41.         while (less(v, a[--j])){
42.             if(j == lo) break;
43.         } // 从右向左找到第一个小于等于基准的元素
44.
45.         if (i >= j) break; // 如果两个指针交叉，停止
46.
47.         exch(a, i, j); // 交换两个元素
48.     }
49.     exch(a, lo, j); // 将基准元素放到正确的位置
50.     return j; // 返回基准元素的位置
51. }
52.
53. // 辅助方法，用于比较两个元素
54. private static boolean less(Comparable v, Comparable w) {
55.     return v.compareTo(w) < 0;
56. }
57.
58. // 交换数组中的两个元素
59. private static void exch(Comparable[] a, int i, int j) {
60.     Comparable swap = a[i];
61.     a[i] = a[j];
62.     a[j] = swap;
63. }
64.
65. // 测试时间
66. public static double time(Comparable[] a) {
67.     StdOut.println("随机快速排序:");
68.     for (int i = 0; i < 10; i++) {
```

```

69.         Comparable[] A = a.clone();
70.
71.         long StartTime = System.nanoTime();
72.         RandomQuicksort.sort(A);
73.         long endTime = System.nanoTime();
74.
75.         StdOut.println((endTime - StartTime) / 1000000.0);
76.     }
77.     StdOut.println(" ");
78.     return 0;
79. }
80.
81. // 测试空间
82. public static double memo(Comparable[] a) {
83.     StdOut.println("随机快速排序:");
84.     for (int i = 0; i < 10; i++) {
85.         Comparable[] B = a.clone();
86.
87.         Runtime run = Runtime.getRuntime();
88.         run.gc();
89.         long startMemo = run.totalMemory() - run.freeMemory();
90.         RandomQuicksort.sort(B);
91.         long endMemo = run.totalMemory() - run.freeMemory();
92.
93.         StdOut.println((endMemo - startMemo) / 1024);
94.     }
95.     return 0;
96. }
97. }

```

5、QuicksortWith3WayPartition

```

1. import edu.princeton.cs.algs4.StdOut;
2.
3. public class QuicksortWith3WayPartition {
4.
5.     // 3-way partitioning
6.     private static void sort(Comparable[] a, int lo, int hi) {
7.         if (hi <= lo) {
8.             return;
9.         }
10.        int lt = lo, i = lo + 1, gt = hi;
11.        Comparable v = a[lo];
12.        while (i <= gt) {
13.            int cmp = a[i].compareTo(v);

```

```

14.         if (cmp < 0) {
15.             exch(a, lt++, i++);
16.         } else if (cmp > 0) {
17.             exch(a, i, gt--);
18.         } else {
19.             i++;
20.         }
21.     }
22.     for (i = lo; i < lt; i++) {
23.         a[i] = v;
24.     }
25.     sort(a, lo, lt - 1);
26.     sort(a, gt + 1, hi);
27. }
28.
29. //交换数组中的两个元素
30. private static void exch(Comparable[] a, int i, int j) {
31.     Comparable temp = a[i];
32.     a[i] = a[j];
33.     a[j] = temp;
34. }
35.
36. // 测试时间
37. public static double time(Comparable[] a) {
38.     StdOut.println("Dijkstra 3-路划分快速排序:");
39.     for (int i = 0; i < 10; i++) {
40.         Comparable[] A = a.clone();
41.
42.         long StartTime = System.nanoTime();
43.         QuicksortWith3WayPartition.sort(A, 0, A.length - 1);
44.         long endTime = System.nanoTime();
45.
46.         StdOut.println((endTime - StartTime) / 1000000.0);
47.     }
48.     StdOut.println(" ");
49.     return 0;
50. }
51.
52. // 测试空间
53. public static double memo(Comparable[] a) {
54.     StdOut.println("Dijkstra 3-路划分快速排序:");
55.     for (int i = 0; i < 10; i++) {
56.         Comparable[] B = a.clone();
57.

```

```

58.         Runtime run = Runtime.getRuntime();
59.         run.gc();
60.         long startMemo = run.totalMemory() - run.freeMemory();
61.         QuicksortWith3WayPartition.sort(B, 0, B.length - 1);
62.         long endMemo = run.totalMemory() - run.freeMemory();
63.
64.         StdOut.println((endMemo - startMemo) / 1024);
65.     }
66.     return 0;
67. }
68. }

```

6、SortCompare.java

```

1. import edu.princeton.cs.algs4.StdIn;
2. import edu.princeton.cs.algs4.StdOut;
3. import edu.princeton.cs.algs4.StdRandom;
4.
5. import java.util.Arrays;
6.
7. public class SortCompare {
8.     public static void main(String[] args) {
9.         StdOut.println("请输入待排序数量 N: ");
10.        int N = StdIn.readInt();
11.        Comparable array[] = new Comparable[N];
12.
13.        for (int i = 0; i < N; i++) {
14.            array[i] = StdRandom.uniform(100);
15.        }
16.
17.        // 测试时间
18.        InsertionSort.time(array);
19.        TopDownMergesort.time(array);
20.        BottomUpMergesort.time(array);
21.        RandomQuicksort.time(array);
22.        QuicksortWith3WayPartition.time(array);
23.
24.        //测试空间
25.        InsertionSort.memo(array);
26.        TopDownMergesort.memo(array);
27.        BottomUpMergesort.memo(array);
28.        RandomQuicksort.memo(array);
29.        QuicksortWith3WayPartition.memo(array);
30.    }
31. }

```

五、 实验结果记录

1、 数据量为 1000

数据量: 1000	数据范围: 0-10000										
time	Run1	Run2	Run3	Run4	Run5	Run6	Run7	Run8	Run9	Run10	Average
IS	1.1163	1.3736	1.3897	2.4292	1.4635	1.6789	5.6898	2.109	1.7909	1.2973	2.0338
TDM	0.7372	0.1668	0.1532	0.154	0.1459	0.1446	0.1476	0.1436	0.1434	0.1459	0.2082
BUM	0.4566	0.1945	0.1848	0.1843	0.1825	0.1821	0.1836	0.238	0.192	0.191	0.2189
RQ	0.6459	0.3073	0.2726	0.1726	0.1653	0.1859	0.2181	0.1699	0.183	0.2042	0.2525
QD3P	0.3172	0.1303	0.3245	0.173	0.2959	0.0599	0.0417	0.0657	0.0426	0.066	0.1517

数据量: 1000	数据范围: 0-10000										
memo	Run1	Run2	Run3	Run4	Run5	Run6	Run7	Run8	Run9	Run10	Average
IS	0	0	0	0	0	0	0	0	0	0	0
TDM	6	6	6	6	6	6	6	6	6	6	6
BUM	6	6	6	6	6	6	6	6	6	6	6
RQ	0	0	0	0	0	0	0	0	0	0	0
QD3P	0	0	0	0	0	0	0	0	0	0	0

2、 数据量为 10000

数据量: 10000	数据范围: 0-10000										
time	Run1	Run2	Run3	Run4	Run5	Run6	Run7	Run8	Run9	Run10	Average
IS	51.1097	49.9216	52.1934	51.8679	51.8898	57.6695	51.2712	56.9895	60.366	57.8167	54.1095
TDM	3.5244	2.4424	2.4548	2.529	2.5089	2.573	3.0749	2.2086	2.1753	2.3632	2.5855
BUM	2.6718	2.4171	2.6125	2.8108	2.7786	2.9029	2.6534	2.2856	2.232	2.2242	2.5589
RQ	2.8656	2.0207	1.9782	2.0377	2.3531	2.0758	9.4521	3.8566	3.7373	4.1337	3.4511
QD3P	1.283	1.5767	0.9083	1.1594	0.9366	0.9508	1.0033	0.9739	0.9803	1.0323	1.0805

数据量: 10000	数据范围: 0-10000										
memo	Run1	Run2	Run3	Run4	Run5	Run6	Run7	Run8	Run9	Run10	Average
IS	0	0	0	0	0	0	0	0	0	0	0
TDM	120	120	120	120	120	120	120	120	120	120	120
BUM	120	120	120	120	120	120	120	120	120	120	120
RQ	0	0	0	0	0	0	0	0	0	0	0
QD3P	0	0	0	0	0	0	0	0	0	0	0

六、 回答问题

1、 Which sort worked best on data in constant or increasing order (i.e., already sorted data)? Why do you think this sort worked best?

答: 快速排序, 因为它只需要遍历一遍数组, 将每个元素插入到已排序部分的正确位置。

2、 Did the same sort do well on the case of mostly sorted data? Why or why not?

答: 初始数据基本有序的情况对各种排序的影响很大, 比如快排的时候, 初始有序的时候就变成 N^2 的算法

3、 In general, did the ordering of the incoming data affect the performance of the sorting algorithms? Please answer this question by referencing specific data from your table to support your answer.

答: 初始状况对于每一种排序的影响是巨大的, 比如我使用均匀分布的十个数据, 但是总数据量有 10000 的时候, 三向切分需要 0.3ms, 而快排此时就需要 1.6ms, 差距为 5 倍; 而当我使用数据范围为 0-10000 的数时, 三向切分需要 1ms, 而快排此时就需要 3.4ms, 差距为 3 倍; 原因是快排处理大量重复数据的时候是非常吃力的。

4、 Which sort did best on the shorter (i.e., $n = 1,000$) data sets? Did the same one do better on the longer (i.e., $n = 100,000$) data sets? Why or why not? Please use specific data from your table to support your answer.

答：如前面表格结果所示：对于少量 ($n=1000$) 数据，快速排序算法在时间占用上表现突出；对于大量 ($n=10000$) 数据，快速排序乱序时算法依旧表现突出。

5、 In general, which sort did better? Give a hypothesis as to why the difference in performance exists.

答：3 路划分快速排序

6、 Are there results in your table that seem to be inconsistent? (e.g., If I get run times for a sort that look like this {1.3, 1.5, 1.6, 7.0, 1.2, 1.6, 1.4, 1.8, 2.0, 1.5] the 7.0 entry is not consistent with the rest). Why do you think this happened?

答：对于数据突然出现的偶然性情况，我认为应该是初始化随机数的时候造成了一些偶然情况的发生，使得初始的排序出现了某种比较差的情况。

实验三：地图路由

一、实验目的

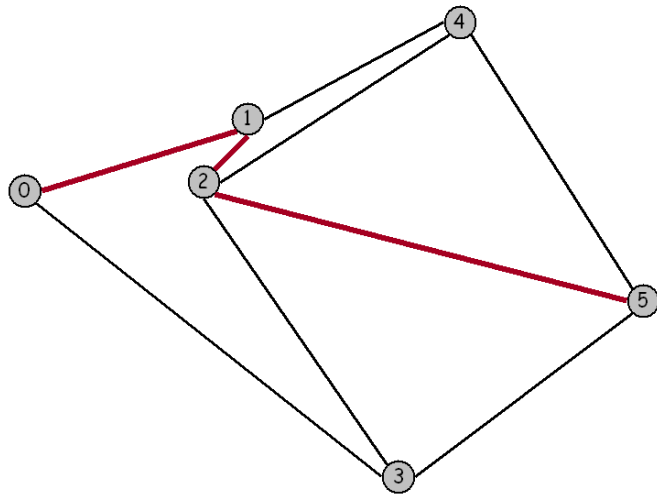
实现经典的 Dijkstra 最短路径算法，并对其进行优化。这种算法广泛应用于地理信息系统 (GIS)，包括 MapQuest 和基于 GPS 的汽车导航系统。

二、实验内容

本次实验对象是图 maps 或 graphs，其中顶点为平面上的点，这些点由权值为欧氏距离的边相连成图。可将顶点视为城市，将边视为相连的道路。为了在文件中表示地图，我们列出了顶点数和边数，然后列出顶点（索引后跟其 x 和 y 坐标），然后列出边（顶点对），最后列出源点和汇点。

```
6 9
0 1000 2400
1 2800 3000
2 2400 2500
3 4000 0
4 4500 3800
5 6000 1500

0 1
0 3
1 2
1 4
2 4
2 3
2 5
3 5
4 5
0 5
```



Dijkstra 算法是最短路径问题的经典解决方案。对于图中的每个顶点，我们维护从源点到该顶点的最短已知的路径长度，并且将这些长度保持在优先队列（priority queue, PQ）中。初始时，我们把所有的顶点放在这个队列中，并设置高优先级，然后将源点的优先级设为 0.0。算法通过从 PQ 中取出最低优先级的顶点，然后检查可从该顶点经由一条边可达的所有顶点，以查看这条边是否提供了从源点到那个顶点较之之前已知的最短路径的更短路径。如果是这样，它会降低优先级来反映这种新的信息。

该方法计算最短路径的长度。为了记录路径，我们还保持每个顶点的源点到该顶点最短路径上的前驱。文件 Euclidean Graph.java, Point.java, IndexPQ.java, IntIterator.java 和 Dijkstra.java 提供了针对 map 的 Dijkstra 算法的基本框架实现，你可以以此作为起点。客户端程序 ShortestPath.java 求解一个单源点最短路径问题，并使用图形绘制了结果。客户端程序 Paths.java 求解了许多最短路径问题，并将最短路径打印到标准输出。

本次实验，我们需要优化 Dijkstra 算法，使其可以处理给定图的数千条最短路径查询。一旦你读取图（并可选地预处理），你的程序应该在亚线性时间内解决最短路径问题。优化的目标是减少每次最短路径计算所涉及的工作量，而不会占用过多的空间。

三、 优化方法

1、 减少检查的顶点数量

Dijkstra 算法的朴素实现检查图中的所有 V 个顶点。减少检查的顶点数量的一种策略是一旦发现目的地的最短路径就停止搜索。通过这种方法，可以使每个最短路径查询的运行时间与 $E' \log V'$ 成比例，其中 E' 和 V' 是 Dijkstra 算法检查的边和顶点数。程序实现如下：

```
1. // 计算并判断最短路径
2. public boolean hasPathTo() {
3.     if(distTo[d] < Double.POSITIVE_INFINITY)
4.     {
5.         return true;
6.     }
7.
8.     while (!pq.isEmpty()) {
9.         int x = pq.delMin();
10.        // 优化 1: 减少检查点的数量
11.        if (x == d) {
12.            return true;
13.        }
14.
15.        for (Edge e : G.adj(x))
16.        {
17.            relax(e, x);
18.        }
19.    }
20.
21.    return distTo[d] < Double.POSITIVE_INFINITY;
22. }
```

2、 欧氏距离松弛边

我们可以利用问题的欧式几何来进一步减少搜索时间。对于一般图，常规的 Dijkstra 算法通过将 $d[w]$ 更新为 $d[v] +$ 从 v 到 w 的距离来松弛边 $v-w$ 。对于地图，则将 $d[w]$ 更新为 $d[v] +$ 从 v 到 w 的距离 $+$ 从 w 到 d 的欧式距离 $-$ 从 v 到 d 的欧式距离。这种方法称之为 A*算法。程序实现如下：

```
1. private void relax(Edge e, int v) {
2.     int w = e.other(v);
3.     // 优化 2: A*算法
4.     double weight = distTo[v] + e.weight() + Nodes[v].Dist(Nodes[d])
5.         - Nodes[w].Dist(Nodes[d]); // 优化 2 进行优化
6.     // double weight = distTo[v] + e.weight();
7.     if (distTo[w] > weight) {
8.         distTo[w] = weight;
9.         edgeTo[w] = e;
10.    }
```

```

9.         if (pq.contains(w)) pq.decreaseKey(w, distTo[w]);
10.        else pq.insert(w, distTo[w]);
11.    }
12. }

```

3、使用多路优先堆

我们也可以考虑使用更快的优先队列作为基础数据结构。本次实验中我选择使用 Sedgwick 在教材中提出的多路最小优先堆（IndexMultiwayMinPQ）。程序实现如下：

```

1. public DijkstraUndirectedSP_3(EdgeWeightedGraph G, Node[] Nodes, int
   s, int d) {
2.     this.G = G;
3.     this.Nodes = Nodes;
4.     this.s = s;
5.     this.d = d;
6.
7.     // 检测边的权重有效
8.     for (Edge e : G.edges()) {
9.         if (e.weight() < 0)
10.            throw new IllegalArgumentException("edge " + e + " has ne
               gative weight");
11.     }
12.
13.     // 开辟空间
14.     distTo = new double[G.V()];
15.     edgeTo = new Edge[G.V()];
16.
17.     // 建立优先队列
18.     // 优化 3: 优化优先队列
19.     //pq = new indexPQ<>(G.V());
20.     pq = new IndexMultiwayMinPQ<Double>(G.V(), 2);
21.
22.     // 初始化源点到其他点之间的距离为无穷
23.     for (int v = 0; v < G.V(); v++)
24.     {
25.         if (pq.contains(v)) {
26.             pq.delete(v);
27.         }
28.         if (edgeTo[v] != null) {
29.             edgeTo[v] = null;
30.         }
31.
32.         distTo[v] = Double.POSITIVE_INFINITY;
33.     }

```

```

34.     distTo[s] = 0.0;
35.
36.     // 插入源点
37.     pq.insert(s, distTo[s]);
38.
39.     // 计算最短路径
40.     hasPathTo();
41.
42.     // 打印最短路径
43.     printPathTo();
44. }

```

四、 源代码

下附代码使用了全优化（即上述三种方法全部优化）的方法进行优化。其他优化方法只需参照上述的优化方法对代码稍作改动即可。

1、DijkstraUndirectedSP_all.java

```

1.  import edu.princeton.cs.algs4.Edge;
2.  import edu.princeton.cs.algs4.EdgeWeightedGraph;
3.  import edu.princeton.cs.algs4.StdOut;
4.  import edu.princeton.cs.algs4.Stack;
5.  import edu.princeton.cs.algs4.IndexMultiwayMinPQ;
6.
7.  public class DijkstraUndirectedSP_all {
8.      private double[] distTo;           // distTo[v] = distance of sho
      rtest s->v path
9.      private Edge[] edgeTo;             // edgeTo[v] = last edge on sho
      rtest s->v path
10.     //private IndexPQ<Double> pq;;    // priority queue of vertices
11.     private IndexMultiwayMinPQ<Double> pq;
12.     private EdgeWeightedGraph G;
13.     private Node[] Nodes;
14.     int s;
15.     int d;
16.
17.     public DijkstraUndirectedSP_all(EdgeWeightedGraph G, Node[] Nodes
        , int s, int d) {
18.         this.G = G;
19.         this.Nodes = Nodes;
20.         this.s = s;
21.         this.d = d;
22.
23.         // 检测边的权重有效
24.         for (Edge e : G.edges()) {

```

```

25.         if (e.weight() < 0)
26.             throw new IllegalArgumentException("edge " + e + " has negative weight");
27.     }
28.
29.     // 开辟空间
30.     distTo = new double[G.V()];
31.     edgeTo = new Edge[G.V()];
32.
33.     // 建立优先队列
34.     // 优化 3: 优化优先队列
35.     //pq = new IndexPQ<>(G.V());
36.     pq = new IndexMultiwayMinPQ<Double>(G.V(), 2);
37.
38.     // 初始化源点到其他点之间的距离为无穷
39.     for (int v = 0; v < G.V(); v++)
40.     {
41.         if (pq.contains(v)) {
42.             pq.delete(v);
43.         }
44.         if (edgeTo[v] != null) {
45.             edgeTo[v] = null;
46.         }
47.
48.         distTo[v] = Double.POSITIVE_INFINITY;
49.     }
50.     distTo[s] = 0.0;
51.
52.     // 插入源点
53.     pq.insert(s, distTo[s]);
54.
55.     // 计算最短路径
56.     hasPathTo();
57.
58.     // 打印最短路径
59.     //printPathTo();
60. }
61.
62. // relax edge e and update pq if changed
63. private void relax(Edge e, int v) {
64.     int w = e.other(v);
65.     //StdOut.println(w);
66.     // 优化 2: A*算法

```

```

67.         double weight = distTo[v] + e.weight() + Nodes[v].Dist(Nodes[
           d]) - Nodes[w].Dist(Nodes[d]); //优化 2 进行优化
68.         //double weight = distTo[v] + e.weight();
69.         if (distTo[w] > weight) {
70.             distTo[w] = weight;
71.             edgeTo[w] = e;
72.             if (pq.contains(w)) pq.decreaseKey(w, distTo[w]);
73.             else pq.insert(w, distTo[w]);
74.         }
75.     }
76.
77.     // 计算并判断最短路径
78.     public boolean hasPathTo() {
79.         if(distTo[d] < Double.POSITIVE_INFINITY)
80.         {
81.             return true;
82.         }
83.
84.         long num = 0;
85.         while (!pq.isEmpty()) {
86.             int x = pq.delMin();
87.             //StdOut.printf("%d\n", x);
88.
89.             num++;
90.             // 优化 1: 减少检查点的数量
91.             if (x == d) {
92.                 //StdOut.printf("%d\n", num);
93.                 //StdOut.printf("1\n");
94.                 return true;
95.             }
96.
97.             for (Edge e : G.adj(x))
98.             {
99.                 relax(e, x);
100.            }
101.        }
102.        //StdOut.printf("%d\n", num);
103.        //StdOut.printf("2\n");
104.        return distTo[d] < Double.POSITIVE_INFINITY;
105.    }
106.
107.    public Iterable<Edge> pathTo() {
108.        if (!hasPathTo()) return null;
109.        Stack<Edge> path = new Stack<Edge>();

```

```

110.         int x = d;
111.         for (Edge e = edgeTo[d]; e != null; e = edgeTo[x]) {
112.             path.push(e);
113.             x = e.other(x);
114.         }
115.         return path;
116.     }
117.
118.     // 打印路径
119.     public void printPathTo() {
120.         if (!hasPathTo()) {
121.             StdOut.printf("没有从 ( %d ) 到 ( %d ) 的路
                径 ", s, d);
122.             return;
123.         }
124.
125.         StdOut.printf("从 ( %d ) 到 ( %d ) 的最短路径: \n", s, d);
126.         int last_to = s;
127.         double sum = 0.0;
128.         for (Edge e : pathTo()) {
129.             int from = e.either();
130.             int to = e.other(from);
131.             double weight = e.weight();
132.             sum += weight;
133.             if(to == last_to)
134.             {
135.                 StdOut.printf("从 %d 到 %d, 路
                    程: %.2f\n", to, from, weight);
136.                 last_to = from;
137.             }
138.             else
139.             {
140.                 StdOut.printf("从 %d 到 %d, 路
                    程: %.2f\n", from, to, weight);
141.                 last_to = to;
142.             }
143.         }
144.         StdOut.printf("总路程: %.2f\n", sum);
145.     }
146. }

```

2、Map. java

```

1. import edu.princeton.cs.algs4.Edge;
2. import edu.princeton.cs.algs4.EdgeWeightedGraph;

```

```

3. import edu.princeton.cs.algs4.In;
4. import edu.princeton.cs.algs4.StdOut;
5. import java.util.Scanner;
6.
7. public class Map {
8.     // 读取文件中节点信息
9.     public static void createPoint(In in, Node[] Nodes, int num_of_v)
10.    {
11.        for (int i = 0; i < num_of_v; i++) {
12.            int n = in.readInt();
13.            int x = in.readInt();
14.            int y = in.readInt();
15.            Nodes[n] = new Node(x, y);
16.        }
17.
18.        // 读取文件中边的信息，并根据节点信息创建图
19.        public static void createGraph(In in, EdgeWeightedGraph Graph, int
20.            num_of_e, Node[] Nodes) {
21.            for (int i = 0; i < num_of_e; i++) {
22.                int n1 = in.readInt();
23.                int n2 = in.readInt();
24.                double weight = Nodes[n1].Dist(Nodes[n2]);
25.                Graph.addEdge(new Edge(n1, n2, weight));
26.            }
27.
28.
29.
30.        public static void main(String[] args) {
31.            In in = new In("E:\\课程任务\\各课程实验\\算法课实验\\(3)地图路
32.                由\\dijk\\usamap.txt");
33.
34.            // 读入节点数和边数
35.            int num_of_v = in.readInt();
36.            int num_of_e = in.readInt();
37.
38.            // 存储节点信息（编号、坐标）
39.            Node[] Nodes = new Node[num_of_v];
40.            createPoint(in, Nodes, num_of_v);
41.
42.            // 读取边的信息，构建图
43.            EdgeWeightedGraph Graph = new EdgeWeightedGraph(num_of_v);
44.            createGraph(in, Graph, num_of_e, Nodes);

```

```
44.
45.     Scanner in2 = new Scanner(System.in);
46.     int s = in2.nextInt();
47.     int d = in2.nextInt();
48.
49.     // 未优化
50.     long StartTime0 = System.nanoTime();
51.     DijkstraUndirectedSP_0 Dijk_0 = new DijkstraUndirectedSP_0(Graph, Nodes, s, d);
52.     long endTime0 = System.nanoTime();
53.     StdOut.println(" ( DijkstraUndirectedSP_0 ) 查询时间占用:" + (endTime0 - StartTime0) / 1000000.0 + " ms");
54.
55.     // 优化 1
56.     long StartTime1 = System.nanoTime();
57.     DijkstraUndirectedSP_1 Dijk_1 = new DijkstraUndirectedSP_1(Graph, Nodes, s, d);
58.     long endTime1 = System.nanoTime();
59.     StdOut.println(" ( DijkstraUndirectedSP_1 ) 查询时间占用:" + (endTime1 - StartTime1) / 1000000.0 + " ms");
60.
61.     // 优化 2
62.     long StartTime2 = System.nanoTime();
63.     DijkstraUndirectedSP_2 Dijk_2 = new DijkstraUndirectedSP_2(Graph, Nodes, s, d);
64.     long endTime2 = System.nanoTime();
65.     StdOut.println(" ( DijkstraUndirectedSP_2 ) 查询时间占用:" + (endTime2 - StartTime2) / 1000000.0 + " ms");
66.
67.     // 优化 3
68.     long StartTime3 = System.nanoTime();
69.     DijkstraUndirectedSP_3 Dijk_3 = new DijkstraUndirectedSP_3(Graph, Nodes, s, d);
70.     long endTime3 = System.nanoTime();
71.     StdOut.println(" ( DijkstraUndirectedSP_3 ) 查询时间占用:" + (endTime3 - StartTime3) / 1000000.0 + " ms");
72.
73.     // 打印路径
74.     Dijk_3.printPathTo();
75.
76.     // 记录数据
77.     StdOut.println((endTime0 - StartTime0) / 1000000.0);
78.     StdOut.println((endTime1 - StartTime1) / 1000000.0);
79.     StdOut.println((endTime2 - StartTime2) / 1000000.0);
```

```
80.         StdOut.println((endTime3 - StartTime3) / 1000000.0);
81.     }
82. }
83.
84. // 定义节点类
85. class Node{
86.     private double x;
87.     private double y;
88.
89.     public Node(double x, double y) {
90.         this.x = x;
91.         this.y = y;
92.     }
93.
94.     // 计算节点之间距离
95.     public double Dist(Node n) { // 计算距离
96.         double delta_x = this.x - n.x;
97.         double delta_y = this.y - n.y;
98.         return Math.sqrt(delta_x * delta_x + delta_y * delta_y);
99.     }
100. }
```

五、实验结果

检索从节点 0 到节点 85574 的路程，记录查询所需时间，记录数据如下：

time(ms)	Run1	Run2	Run3	Run4	Run5	Run6	Run7	Run8	Run9	Run10	Average
无优化	52.92	57.13	58.07	55.07	56.32	57.47	57.91	57.64	58.34	56.91	56.78
优化1	42.93	41.72	42.80	40.82	40.70	39.08	40.80	38.92	40.16	38.59	40.65
优化2	41.11	40.76	40.08	38.17	38.29	38.40	39.62	38.94	41.76	39.12	39.63
优化3	53.42	55.52	55.88	53.87	52.72	56.87	50.16	53.88	53.48	53.86	53.97
全优化	39.58	34.51	36.02	36.28	36.91	34.49	34.34	36.64	35.65	34.32	35.87

从实验结果可以看出，不同的优化策略对 Dijkstra 算法性能有着明显的影响。特别是当将多个优化方案结合起来时（如全优化），可以显著提高算法效率。

六、实验总结

地图路由实验旨在通过实现和优化 Dijkstra 最短路径算法处理大规模图上的最短路径查询问题。根据实验结果，我们观察到了不同优化策略对算法执行时间的影响。无优化的原始 Dijkstra 算法在测试环境中表现出了较高的延迟，而通过一系列的优化措施，如提前终止搜索、应用 A*启发式搜索以及使用更快的数据结构等，最终实现了显著的时间节省。

实验四：Seam Carving

一、实验目的

实现并深入理解一种先进的内容感知图像调整技术——接缝雕刻（Seam Carving）。通过这次实验，我们不仅要掌握接缝雕刻的基本原理和算法细节，还要能够应用这一技术来智能地减少图像的尺寸，同时最大限度地保留图像的关键特征，如宽高比、图像中的对象等。这与传统的裁剪或缩放方法不同，后者通常不考虑图像内容的重要性。

二、实验内容

缝线雕刻是一种内容感知图像大小调整技术，每次将图像大小缩小一个像素的高度（或宽度）。图像中的垂直缝是自上至下连接的像素路径，每一行有一个像素；水平接缝是从左到右连接的像素路径，每列有一个像素。左下是 505×287 像素的原始图像；右下图是去除 150 条垂直接缝后的结果，图像缩小了 30%。与标准的与内容无关的调整大小技术（如裁剪和缩放）不同，接缝雕刻保留了图像中最感兴趣的特征（长宽比、呈现的对象集等）。



在本作业中，将创建一个数据类型，使用缝雕刻技术调整 $w \times h$ 图像的大小。
查找和删除接缝

这涉及三个步骤：

(1) 能量计算 (Energy calculation):

实验的第一个步骤是计算图像中每个像素的“能量”，这是一种衡量像素重要性的指标。能量越高的像素，越不可能被包含在 seam 中。使用“双梯度能量函数” (dual-gradient energy function) 来计算像素的能量，该函数基于图像中颜色梯度的快速变化来确定像素的能量值。能量计算方法如下：

对于像素 (x, y) ，其能量 $E(x, y)$ 计算如下：

$$E(x, y) = \sqrt{\Delta_x^2(x, y) + \Delta_y^2(x, y)}$$

其中， $\Delta_x^2(x, y)$ 是 x 方向（水平）的梯度平方和， $\Delta_y^2(x, y)$ 是 y 方向（垂直）的梯度平方和。

$$\Delta_x^2(x, y) = R_x(x, y)^2 + G_x(x, y)^2 + B_x(x, y)^2$$

$$\Delta_y^2(x, y) = R_y(x, y)^2 + G_y(x, y)^2 + B_y(x, y)^2$$

这里， $R_x(x, y)$ 、 $G_x(x, y)$ 、 $B_x(x, y)$ 分别是红、绿、蓝颜色通道在 x 方向上的差异， $R_y(x, y)$ 、 $G_y(x, y)$ 、 $B_y(x, y)$ 分别是红、绿、蓝颜色通道在 y 方向上的差异。

(2) Seam 识别 (Seam identification):

接下来需要找到一条垂直 seam，其总能量最小。这个过程类似于最短路径问题，但是有几个关键的不同点：

- 权重在顶点上而不是边上。
- 需要从顶部的任意像素找到到底部任意像素的最短路径。
- 图像是无环的，并且有向下的边从像素 (x, y) 指向像素 $(x-1, y+1)$ ， $(x, y+1)$ ，和 $(x+1, y+1)$ 。

(3) Seam 移除 (Seam removal):

最后一步是从图像中移除垂直或水平 seam 上的所有像素。

实验要求实现一个可变的 SeamCarver 数据类型，包含以下方法：

```
SeamCarver(Picture picture): 构造函数。
Picture picture(): 返回原始图像。
int width(): 返回图像宽度。
int height(): 返回图像高度。
double energy(int x, int y): 计算给定坐标的像素能量。
int[] findHorizontalSeam(): 找到并返回水平 seam。
int[] findVerticalSeam(): 找到并返回垂直 seam。
void removeHorizontalSeam(int[] seam): 移除水平 seam。
void removeVerticalSeam(int[] seam): 移除垂直 seam。
```

三、具体实现

1、SeamCarver 函数

```
1. public SeamCarver(Picture picture)
2.     {
3.         // 检测图片有效性
4.         if (picture == null) {
5.             throw new IllegalArgumentException("Picture cannot be null");
6.         }
7.
8.         // 获取图片
9.         Original_Picture = picture;
10.        width = Original_Picture.width();
11.        height = Original_Picture.height();
12.
13.        // 获取图片各像素的 RGB 信息，存储到 Picture_RGB 中
14.        Picture_RGB = new int[height][width];
15.        get_Picture_RGB();
16.
17.        // 计算能量图
18.        Picture_Energy = new double[height][width];
19.        get_Picture_energy();
```

```
20.  
21.     Vertical_path = new int[height];  
22.     Horizontal_path = new int[width];  
23. }
```

2、Picture 函数

```
1. // 获取当前图片  
2. public Picture picture()  
3. {  
4.     Final_Picture = new Picture(width, height);  
5.     for (int y = 0; y < height; y++) {  
6.         for (int x = 0; x < width; x++) {  
7.             Final_Picture.setRGB(x, y, Picture_RGB[y][x]);  
8.         }  
9.     }  
10.  
11.     return Final_Picture;  
12. }
```

3、Width 函数和 Height 函数

```
1. public int width() {return this.width;}  
2.  
3.     public int height() {return this.height;}
```

4、energy 函数

```
1. // 计算某点的 energy  
2. public double energy(int x, int y)  
3. {  
4.     // 边界检测  
5.     if (x < 0 || x >= width || y < 0 || y >= height) {  
6.         throw new IllegalArgumentException("Coordinates out of range");  
7.     }  
8.  
9.     // 获取该像素点上下左右像素点的 RGB 值  
10.    int left = Picture_RGB[y][(x - 1 + width) % width];  
11.    int right = Picture_RGB[y][(x + 1) % width];  
12.    int up = Picture_RGB[(y - 1 + height) % height][x];  
13.    int down = Picture_RGB[(y + 1) % height][x];  
14.  
15.    // 代入公式计算  
16.    int dxR = getRed(left) - getRed(right);  
17.    int dxG = getGreen(left) - getGreen(right);  
18.    int dxB = getBlue(left) - getBlue(right);
```

```

19.
20.     int dyR = getRed(up) - getRed(down);
21.     int dyG = getGreen(up) - getGreen(down);
22.     int dyB = getBlue(up) - getBlue(down);
23.
24.     int gradX = dxR * dxR + dxG * dxG + dxB * dxB;
25.     int gradY = dyR * dyR + dyG * dyG + dyB * dyB;
26.
27.     return Math.sqrt(gradX + gradY);
28. }

```

5、findHorizontalSeam

```

1.  // 动态规划计算最短水平能量路径
2.  public int[] findHorizontalSeam()
3.  {
4.      double[][] distTo = new double[height][width];
5.      int[][] edgeTo = new int[height][width];
6.
7.      // 初始化第一列的距离为各自能量
8.      for (int y = 0; y < height; y++) {
9.          distTo[y][0] = Picture_Energy[y][0];
10.     }
11.
12.     // 初始化其他列距离为无穷远
13.     for (int x = 1; x < width; x++) {
14.         for (int y = 0; y < height; y++) {
15.             distTo[y][x] = Double.POSITIVE_INFINITY;
16.         }
17.     }
18.
19.     // 动态规划填充 distTo 和 edgeTo 数组
20.     for (int x = 0; x < width - 1; x++) {
21.         for (int y = 0; y < height; y++) {
22.             Horizontal_relax(x, y, x + 1, y - 1, distTo, edgeTo);
23.             // 上右
24.             Horizontal_relax(x, y, x + 1, y, distTo, edgeTo);
25.             // 正右
26.             Horizontal_relax(x, y, x + 1, y + 1, distTo, edgeTo);
27.             // 下右
28.         }
29.     }
30.
31.     // 找到最后一列中能量最小的路径终点
32.     double minDist = Double.MAX_VALUE;
33.     int endPoint = -1;

```

```

31.         for (int y = 0; y < height; y++) {
32.             if (distTo[y][width - 1] < minDist) {
33.                 minDist = distTo[y][width - 1];
34.                 endPoint = y;
35.             }
36.         }
37.
38.         // 回溯路径
39.         Horizontal_path[width - 1] = endPoint;
40.         for (int x = width - 2; x >= 0; x--) {
41.             Horizontal_path[x] = edgeTo[Horizontal_path[x + 1]][x + 1];
42.         }
43.
44.         return Horizontal_path;
45.     }
46.
47.     // 松弛边
48.     private void Horizontal_relax(int xFrom, int yFrom, int xTo, int
        yTo, double[][] distTo, int[][] edgeTo)
49.     {
50.         if (xTo < 0 || xTo >= width || yTo < 0 || yTo >= height) {
51.             return;
52.         }
53.
54.         if (distTo[yFrom][xFrom] + Picture_Energy[yTo][xTo] < distTo[
            yTo][xTo]) {
55.             distTo[yTo][xTo] = distTo[yFrom][xFrom] + Picture_Energy[
                yTo][xTo];
56.             edgeTo[yTo][xTo] = yFrom;
57.         }
58.     }

```

6、findVerticalSeam

```

1.     // 动态规划计算垂直最短能量路径
2.     public int[] findVerticalSeam()
3.     {
4.         double[][] distTo = new double[height][width];
5.         int[][] edgeTo = new int[height][width];
6.
7.         // 初始化第一行的距离为各自能量
8.         for (int x = 0; x < width; x++) {
9.             distTo[0][x] = Picture_Energy[0][x];
10.        }

```

```

11.
12.     // 初始化其他行距离为无穷远
13.     for (int y = 1; y < height; y++) {
14.         for (int x = 0; x < width; x++) {
15.             distTo[y][x] = Double.POSITIVE_INFINITY;
16.         }
17.     }
18.
19.     // 动态规划填充 distTo 和 edgeTo 数组
20.     for (int y = 0; y < height - 1; y++) {
21.         for (int x = 0; x < width; x++) {
22.             Vertical_relax(x, y, x - 1, y + 1, distTo, edgeTo);
23.             Vertical_relax(x, y, x, y + 1, distTo, edgeTo);
24.             Vertical_relax(x, y, x + 1, y + 1, distTo, edgeTo);
25.         }
26.     }
27.
28.     // 找到最后一行中能量最小的路径终点
29.     double minDist = Double.MAX_VALUE;
30.     int endPoint = -1;
31.     for (int x = 0; x < width; x++) {
32.         if (distTo[height - 1][x] < minDist) {
33.             minDist = distTo[height - 1][x];
34.             endPoint = x;
35.         }
36.     }
37.
38.     // 回溯路径
39.     Vertical_path[height - 1] = endPoint;
40.     for (int y = height - 2; y >= 0; y--) {
41.         Vertical_path[y] = edgeTo[y + 1][Vertical_path[y + 1]];
42.     }
43.
44.     return Vertical_path;
45. }
46.
47. // 松弛边
48. private void Vertical_relax(int xFrom, int yFrom, int xTo, int yTo, double[][] distTo, int[][] edgeTo)
49. {
50.     if (xTo < 0 || xTo >= width || yTo < 0 || yTo >= height) {
51.         return;
52.     }
53.

```

```

54.         if (distTo[yFrom][xFrom] + Picture_Energy[yTo][xTo] < distTo[
           yTo][xTo]) {
55.             distTo[yTo][xTo] = distTo[yFrom][xFrom] + Picture_Energy[
           yTo][xTo];
56.             edgeTo[yTo][xTo] = xFrom;
57.         }
58.     }

```

7、removeHorizontalSeam

```

1.     // 移除水平 seam
2.     public void removeHorizontalSeam(int[] seam)
3.     {
4.         if (seam == null) {
5.             throw new IllegalArgumentException("Seam cannot be null")
           ;
6.         }
7.
8.         if (height == 1) {
9.             throw new IllegalArgumentException("Height of the picture
           is 1, cannot remove horizontal seam");
10.        }
11.
12.        validateSeam(seam, true, width);
13.
14.        // 根据 seam 更新 Picture_RGB 的值，可由 Picture_RGB 建图，即为
           新图
15.        for (int x = 0; x < width; x++) {
16.            for (int y = seam[x]; y < height - 1; y++) {
17.                Picture_RGB[y][x] = Picture_RGB[y + 1][x];
18.            }
19.        }
20.
21.        // 更新高度信息
22.        height--;
23.
24.        // 更新能量图
25.        get_Picture_energy();
26.    }

```

8、removeVerticalSeam

```

1.     // 移除垂直 seam
2.     public void removeVerticalSeam(int[] seam)
3.     {
4.         if (seam == null) {

```

```

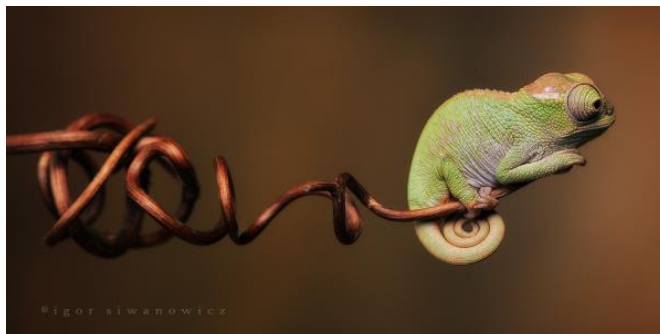
5.         throw new IllegalArgumentException("Seam cannot be null")
        ;
6.     }
7.
8.     if (width == 1) {
9.         throw new IllegalArgumentException("Width of the picture
        is 1, cannot remove vertical seam");
10.    }
11.
12.    validateSeam(seam, false, height);
13.
14.    // 根据 seam 更新 Picture_RGB 的值, 可由 Picture_RGB 建图, 即为
        新图
15.    for (int y = 0; y < height; y++) {
16.        for (int x = seam[y]; x < width - 1; x++) {
17.            Picture_RGB[y][x] = Picture_RGB[y][x + 1];
18.        }
19.    }
20.
21.    // 更新宽度信息
22.    width--;
23.
24.    // 更新能量图
25.    get_Picture_energy();
26.    }

```

四、实验结果

借助实验测试网站提供的 Resize_Demo.java 方法(使用接缝移除方法来调整图像的大小)进行测试

原始图片如下 (600x300):



经过裁剪, 图片如下 (300x200):